

# Control-Flow Graphs and Statement, Branch, and Path Coverage

Week 5 Lecture — SE 413/513: Software Testing

Dr. Kamrul Hasan

Grand Valley State University

Fall 2026

# Learning Objectives

By the end of today's class, you should be able to:

- Explain the difference between **black-box** and **white-box** test design, and where coverage fits.
- Construct a **control-flow graph (CFG)** from a function's source code.
- Define **statement**, **branch**, and **path coverage**, and compute each for a small function.
- Explain why 100% statement coverage does not imply 100% branch coverage.
- Explain why path coverage is often infeasible for code containing loops.
- Compute **cyclomatic complexity** and relate it to the minimum number of tests for branch coverage.
- Measure coverage with `coverage.py` / `pytest-cov`.
- Explain what coverage does and does not tell you about correctness.

# Part 1

## Black-Box vs. White-Box Testing

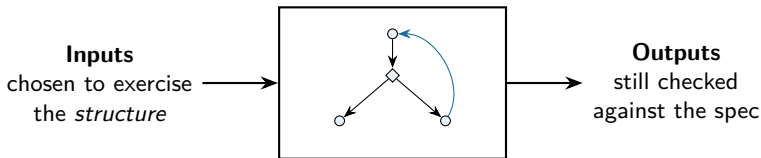
~12 minutes

# Two Ways to Look at the Same Code

## Black-box (specification-based) — Week 3

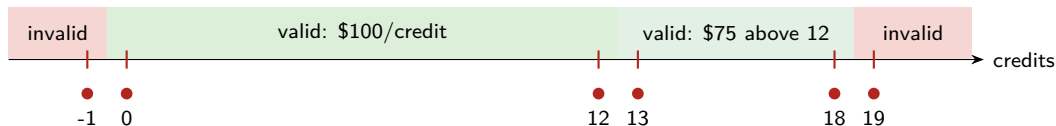


## White-box (structure-based, “glass box”) — today



## Black-Box Recap: The Registration Fee

**From earlier weeks: Week 3, Assignment 1** **Spec:** \$100/credit for the first 12 credits; \$75/credit above 12, up to 18. total\_credits must be 0–18, otherwise ValueError. *We never look at the code.*



red dots = boundary-value test inputs

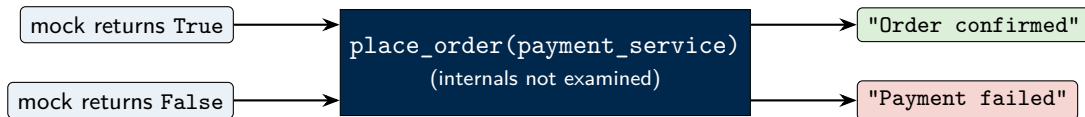
```
@pytest.mark.parametrize("credits, fee", [
    (0, 0), (12, 1200),
    (13, 1275), (18, 1650),
])

def test_fee_boundaries(credits, fee):
    assert calculate_registration_fee(credits)
        == fee
```

```
@pytest.mark.parametrize("credits", [-1, 19])
def test_fee_rejects_invalid(credits):
    with pytest.raises(ValueError):
        calculate_registration_fee(credits)
```

# Black-Box Recap: Testing Through the Interface

From earlier weeks: Week 4



Our Week 4 tests chose inputs (a mocked payment result) and checked outputs (a message) — the **specified behavior**, not the code's structure.

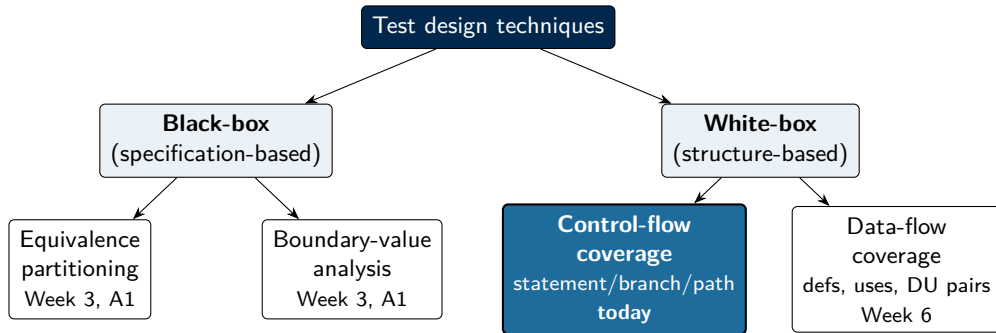
## Key Idea

Black-box design — equivalence partitioning, boundary-value analysis, testing through an interface — asks: *"Have I chosen a good, representative set of inputs based on what this code is supposed to do?"*

# Black-Box vs. White-Box at a Glance

|                       | Black-box                                                                | White-box                                                |
|-----------------------|--------------------------------------------------------------------------|----------------------------------------------------------|
| Tests derived from    | Requirements / specification                                             | Source code structure                                    |
| Need to see the code? | No                                                                       | Yes                                                      |
| Core question         | “Did I pick good, representative inputs for what this <i>should</i> do?” | “Did my tests actually <i>execute</i> the code I wrote?” |
| Techniques            | Equivalence partitioning, boundary-value analysis                        | CFGs; statement, branch, path coverage                   |
| Blind spot            | Code no requirement mentions (dead or extra paths)                       | Missing behavior; exact boundary values                  |
| In this course        | Week 3, Assignment 1                                                     | <b>This week</b> ; data flow next                        |

# Where Today Fits



## Key Idea

A test suite can do very well on one and poorly on the other — and, as we'll see, doing well on *both* is still not the same as being correct.

# Part 2

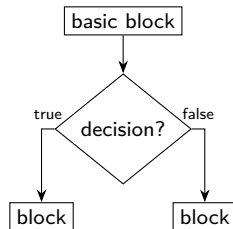
## Control-Flow Graphs

~8 minutes

# What Is a Control-Flow Graph?

A **control-flow graph (CFG)** is a directed graph of a program's possible executions.

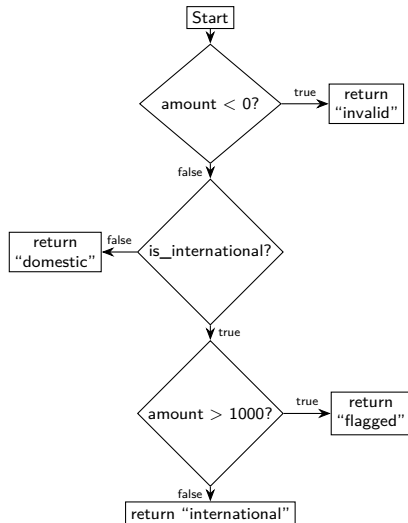
- **Node** = a **basic block**: straight-line statements, no branches in or out except at its start and end.
- **Edge** = a possible transfer of control: falling through, taking a branch, or looping back.
- **Decision node** (diamond): control can leave along two edges — true or false.



# Building a CFG: classify\_transaction()

```
def classify_transaction(amount,
    is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
    else:
        return "domestic"
```

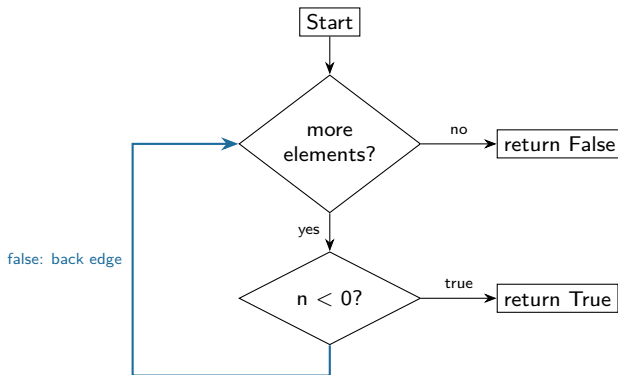
Every if is a decision node. Each root-to-leaf path is one possible execution.



# Loops Add a Back Edge

```
def contains_negative(numbers):  
    for n in numbers:  
        if n < 0:  
            return True  
    return False
```

- The loop condition (“more elements?”) is itself a decision node.
- A **back edge** returns to it from later in the graph.
- Unlike `classify_transaction()`, this graph has a **cycle** — which will matter when we count paths.



# Part 3

## Statement Coverage & the “goto fail” Case

~9 minutes

# Statement Coverage

```
def classify_transaction(  
    amount, is_international):  
  
    if amount < 0:  
        return "invalid"  
    if is_international:  
        if amount > 1000:  
            return "flagged"  
        else:  
            return "  
international"  
    else:  
        return "domestic"
```

Every if is a decision node.  
Each root-to-leaf path is one  
possible execution.

| Test added   | New statements executed                                 | Coverage          |
|--------------|---------------------------------------------------------|-------------------|
| (-5, False)  | amount < 0, return "invalid"                            | 2/7 $\approx$ 29% |
| (50, True)   | is_international, amount > 1000, return "international" | 5/7 $\approx$ 71% |
| (50, False)  | return "domestic"                                       | 6/7 $\approx$ 86% |
| (5000, True) | return "flagged"                                        | 7/7 = 100%        |

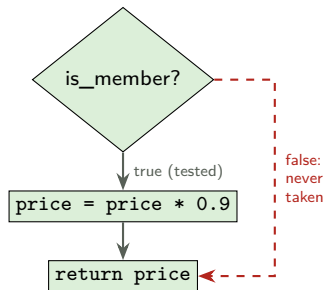
$$\text{Statement coverage} = \frac{\text{statements executed by at least one test}}{\text{total executable statements}} \times 100\%$$

classify\_transaction() has 7 executable statements: 3 if checks + 4 returns.

# The Blind Spot of Statement Coverage

```
def apply_membership_discount(price,
                              is_member):  
    if is_member:  
        price = price * 0.9  
    return price
```

One test, `apply_membership_discount(100, True)`, runs **every statement** — **100% statement coverage**.



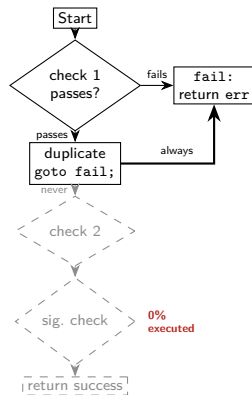
## Common Pitfall

The `is_member=False` path never ran. If a later edit breaks it (e.g., applies the discount unconditionally), this “100%” suite won’t notice.

# Case Study: Apple “goto fail” (CVE-2014-1266)

```
if ((err = SSLHashSHA1.update(
    &hashCtx, &signedParams)) != 0)
    goto fail;
goto fail; // duplicate, NOT in the if
if ((err = SSLHashSHA1.update(
    &hashCtx, &hashOut)) != 0)
    goto fail; // never reached
...
fail:
    return err; // err is still 0 = success
```

SecureTransport (iOS, OS X, Safari) accepted invalid signatures  $\Rightarrow$  man-in-the-middle attacks on TLS.



# What “goto fail” Teaches

- Every check after the duplicate line became **unreachable code** — no input could ever execute it.
- Black-box tests with *valid* certificates all passed: no requirement says “the signature check must be reachable.”
- **Any statement-coverage report** would have shown the signature check at **0% executed**, under every test.

## Tip

This exact bug shape can't happen in Python, where indentation *is* the block structure. But an early `return` that accidentally short-circuits later validation is easy to write in any language — and statement coverage makes it visible immediately.

# Part 4

## Branch (Decision) Coverage

~6 minutes

# Branch Coverage

$$\text{Branch coverage} = \frac{\text{branch outcomes taken by at least one test}}{\text{total branch outcomes}} \times 100\%$$

Back to `apply_membership_discount()` — one decision, two outcomes:

| Test suite                | Statement | Branch |
|---------------------------|-----------|--------|
| (100, True)               | 100%      | 50%    |
| (100, True), (100, False) | 100%      | 100%   |

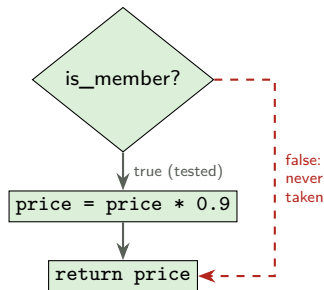
## Key Idea

Branch coverage **subsumes** statement coverage: 100% branch  $\Rightarrow$  100% statement (every statement lies on some branch outcome), but *not* the reverse. That's why most coverage tools report branch coverage too.

# The Blind Spot of Statement Coverage

```
def apply_membership_discount(price,  
                             is_member):  
    if is_member:  
        price = price * 0.9  
    return price
```

One test, `apply_membership_discount(100, True)`, runs **every statement** — **100% statement coverage**.



## Common Pitfall

The `is_member=False` path never ran. If a later edit breaks it (e.g., applies the discount unconditionally), this “100%” suite won’t notice.

# A Useful Habit

For every `if`, `elif`, `while`, and short-circuit `and/or`, ask:

*“Do I have a test that makes this **true** — and a separate test that makes it **false**?”*

## Tip

An `if` with no `else` is the classic place branch coverage catches a gap that statement coverage hides — exactly the `apply_membership_discount()` shape.

## Quick Check — Think, Then Discuss

How many tests does `classify_transaction()` need for 100% branch coverage? Is the 4-test suite from the statement-coverage slide enough?

# Part 5

## Path Coverage, Cyclomatic Complexity & the Hierarchy

~10 minutes

# Path Coverage

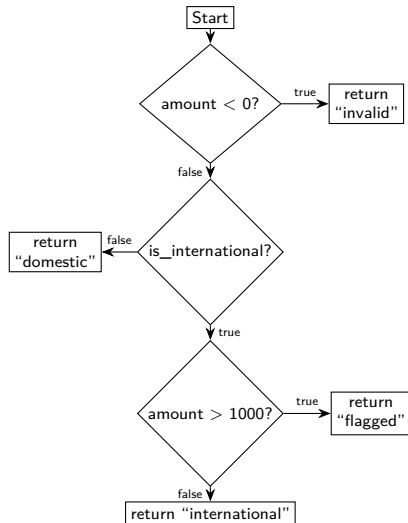
**Path coverage** = % of distinct, complete entry-to-exit paths through the CFG exercised by at least one test. The **strongest** of the three criteria.

- `classify_transaction()` has exactly **4 paths** — the same 4 tests give 100% path coverage, because the graph has **no loops**.
- `contains_negative()` is a different story...

# Building a CFG: classify\_transaction()

```
def classify_transaction(amount,
    is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
    else:
        return "domestic"
```

Every if is a decision node. Each root-to-leaf path is one possible execution.

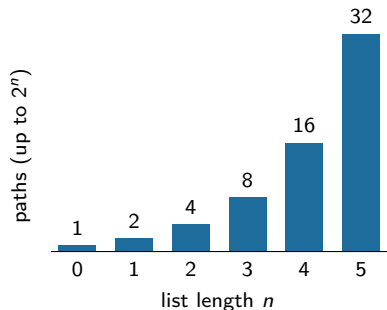


# Why Path Coverage Explodes With Loops

Paths through `contains_negative()` depend on how many times the loop runs:

- 0 iterations (empty list)
- 1 iteration, returns True
- 1 iteration, doesn't
- 2 iterations ... and so on

A list of length  $n$  gives up to  $2^n$  paths. For unbounded input: **unbounded paths**.



## Common Pitfall

Exhaustive path coverage is generally **infeasible** for any code with a loop whose iteration count depends on input.

# Cyclomatic Complexity (McCabe, 1976)

Counts the number of **linearly independent** paths through a CFG:

$$V(G) = E - N + 2 \quad \text{or, for structured code,} \quad V(G) = \text{decision points} + 1$$

| Function                            | Decisions                                                                                  | $V(G)$ |
|-------------------------------------|--------------------------------------------------------------------------------------------|--------|
| <code>classify_transaction()</code> | <code>amount &lt; 0</code> , <code>is_international</code> , <code>amount &gt; 1000</code> | 4      |
| <code>contains_negative()</code>    | loop condition, <code>n &lt; 0</code>                                                      | 3      |

## Tip

$E - N + 2$  assumes a **single exit** node. `classify_transaction()`'s CFG has 4 return nodes; join them into one exit node first (11 edges, 9 nodes  $\Rightarrow$  4). Otherwise the formula gives the wrong answer.

Beyond the reading

# Using $V(G)$ : Basis Path Testing

## Key Idea

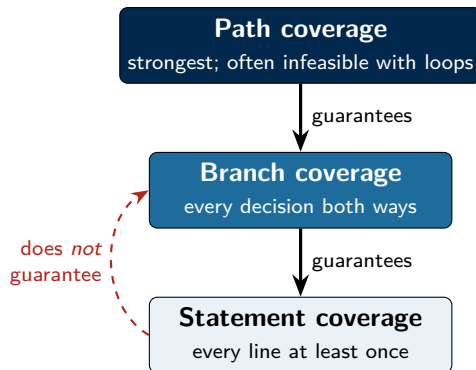
- 1  $V(G)$  is the **minimum** number of tests needed for 100% branch coverage.
- 2 **Basis path testing:** instead of every (possibly infinite) path, choose  $V(G)$  independent paths that together exercise every edge — a practical, finite stand-in for full path coverage.

For `contains_negative()` ( $V(G) = 3$ ), the classic “zero, one, many” idea:

`[]`  $\rightarrow$  False      `[-1]`  $\rightarrow$  True      `[2, -1]`  $\rightarrow$  True (loops once, then finds it)

Maintainability signal:  $V(G)$  above  $\sim 10$  often marks a function worth refactoring.

# The Subsumption Hierarchy



A suite can hit 100% statement while missing branches (`apply_membership_discount`), and 100% branch while missing loop paths (`contains_negative`).

# Part 6

## Measuring Coverage & What It Cannot Tell You

~7 minutes

# Measuring Coverage with pytest-cov

```
pip install pytest-cov
pytest --cov=myapp --cov-branch --cov-report=term-missing
```

Suite that only tests the *member* branch of `apply_membership_discount()`:

| Name     | Stmts | Miss | Branch | BrPart | Cover | Missing |
|----------|-------|------|--------|--------|-------|---------|
| -----    |       |      |        |        |       |         |
| myapp.py | 4     | 0    | 2      | 1      | 83%   | 2->4    |

- Miss 0: every statement ran. BrPart 1: one decision was only *partially* taken.
- 2->4: at line 2 (`if is_member:`), the jump straight to line 4 — the **False** branch — never happened.

## Tip

`-cov-branch` matters: by default `coverage.py` reports **statement coverage only**, which can look deceptively complete. (Output above verified with a real run.)

# What Coverage Cannot Tell You (1): Correctness

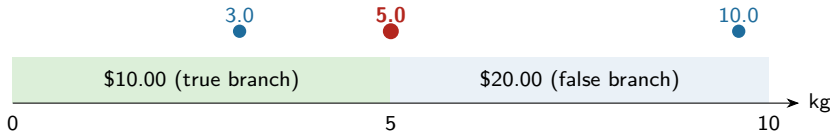
Even 100% path coverage is **not** a correctness guarantee.

- Recall *Testing Foundations*, Principle 1: tests show the *presence* of defects, not their absence.
- Coverage says a line *ran* — not that any test *checked* its result.
- A test with no assertion (or an always-true one) can reach 100% coverage of buggy code while catching nothing.

# What Coverage Cannot Tell You (2): Boundary Values

From earlier weeks: **Week 3** From the *Equivalence Partitioning* reading — `determine_shipping_cost()` has an off-by-one:

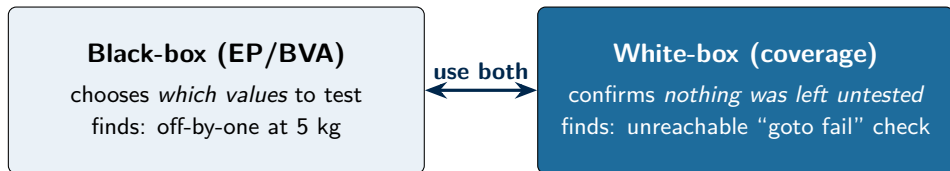
```
elif weight_kg < 5:           # should have been <= 5
    return 10.00
else:
    return 20.00
```



**White-box:** 3.0 and 10.0 take both branches  $\Rightarrow$  **100% branch coverage**. Bug *not* found.

**Black-box (BVA):** the spec's boundary is 5 kg  $\Rightarrow$  test 5.0  $\Rightarrow$  expect \$10.00, get \$20.00. **Bug found.**

# Coverage and EP/BVA Are Complementary



## Key Idea

Coverage answers “did I exercise this code at all?”; EP/BVA answers “did I choose the inputs most likely to expose a defect?” A mature suite uses black-box techniques to choose *which* values, and coverage to confirm nothing in the implementation was left untouched.

# Part 7

## Practice, Pitfalls & Wrap-Up

~5 minutes

# In-Class Practice (Reading Exercises 1 & 3)

```
def grade_letter(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    elif score >= 70:  
        return "C"  
    else:  
        return "F"
```

## Quick Check — Think, Then Discuss

- 1 Sketch the CFG: label each decision node and terminal block.
- 2 What is  $V(G)$  (decision points +1)? How many tests, at minimum, for 100% branch coverage?

2 minutes with a neighbor. Exercises 2, 4, and 5 are homework.

# Common Pitfalls

## Common Pitfall

**Treating a coverage percentage as the goal.** Chasing 100% can produce tests that just execute lines, with weak or missing assertions. Coverage is a diagnostic for gaps.

## Common Pitfall

**Reporting statement coverage as if it were branch coverage.** They can differ a lot on the same code — check which one a number measures.

## Common Pitfall

**Assuming high coverage means boundaries were tested.** Coverage can't tell a boundary test from an arbitrary interior value.

# Key Takeaways

- ❶ **Black-box and white-box are complementary:** one picks *which values* from the spec; the other shows what the *implementation* actually executed.
- ❷ **A CFG makes structure explicit:** basic blocks, decision edges, loop back-edges.
- ❸ **Path  $\Rightarrow$  branch  $\Rightarrow$  statement**, strongest to weakest. Prefer branch over statement.
- ❹ **Loops make full path coverage infeasible;**  $V(G)$  and basis path testing give a finite alternative.
- ❺ **Coverage measures execution, not correctness** — and it cannot see boundary values.
- ❻ **Unreachable code is a real, findable defect:** “goto fail” would have shown up as 0% executed.

# What We Didn't Cover Today

In the interest of pacing, these are in the reading but were **not** covered (or only briefly) in lecture — you are responsible for them as reading:

- Section 11, “A Practical Mental Model” — the four think-aloud prompts (also in backup slides).
- Section 12, the full Summary table (also in backup slides).
- Section 13, Exercises 2, 4, and 5 — **homework** (Exercises 1 and 3 started in class).
- Section 14, Discussion Questions (backup slides list them all).
- Section 16, References — consult the reading directly for citations.

Also note: the registration-fee and `place_order` examples came from Week 3, Week 4, and Assignment 1 — they are not in this reading.

# Questions?

Homework: Reading Exercises 2, 4, and 5

Dr. Kamrul Hasan | Grand Valley State University

# Backup Slides

Reference material only — not part of the 60-minute lecture

# Backup: A Practical Mental Model (Reading, Section 11)

**100% statement coverage?** *“Every line ran. Did every decision’s **both** outcomes run? Check branch coverage.”*

**An if with no else in the tests?** *“A branch-coverage gap waiting to be found — what test drives this false?”*

**A loop?** *“Full path coverage is likely infeasible. What is  $V(G)$ , and what small set of paths (zero, one, many) gives good basis-path coverage?”*

**100% and still suspicious?** *“Coverage confirms the code ran — not that I used the values most likely to expose a defect. Did I also apply boundary-value analysis?”*

## Backup: Summary Table (Reading, Section 12)

| Term                         | Meaning                                                                 |
|------------------------------|-------------------------------------------------------------------------|
| Control-flow graph (CFG)     | Directed graph of basic blocks (nodes) and transfers of control (edges) |
| Basic block                  | Straight-line statements with no internal branches                      |
| Statement coverage           | % of statements executed by at least one test                           |
| Branch coverage              | % of decision outcomes (true/false) executed by at least one test       |
| Path coverage                | % of distinct entry-to-exit paths executed by at least one test         |
| Cyclomatic complexity $V(G)$ | Number of linearly independent paths; decision points + 1               |
| Basis path testing           | Testing $V(G)$ independent paths as a stand-in for full path coverage   |
| Subsumption hierarchy        | Path $\Rightarrow$ branch $\Rightarrow$ statement, strongest to weakest |

# Backup: Remaining Practice Exercises (Homework)

- **Exercise 2:** For `grade_letter()`, give inputs that achieve 100% statement but  $<100\%$  branch coverage; then extend to 100% branch coverage.
- **Exercise 4:** For a “first index of target, or  $-1$ ” function with a `for` loop and early `return`, explain why paths grow with list length, and propose a small, representative (basis-path-style) test set.
- **Exercise 5:** “100% branch coverage, so the code is correct” — argue what this gets right and what it misses, using the hierarchy, the 5 kg boundary example, or “goto fail.”

# Backup: Discussion Questions (Reading, Section 14)

- 1 What is a basic block, and why do CFGs use basic blocks as nodes instead of individual statements?
- 2 Why does 100% statement coverage not guarantee 100% branch coverage? Construct your own example.
- 3 Why is branch coverage said to “subsume” statement coverage?
- 4 Why does a loop make exhaustive path coverage infeasible in general?
- 5 What does cyclomatic complexity actually count, and why is decision points + 1 a useful shortcut?
- 6 How does basis path testing make path-coverage-like guarantees practical for looping code?
- 7 In the “goto fail” case, which coverage criterion most directly would have flagged the defect, and why?
- 8 Why can a coverage tool not detect that a test used a boundary value rather than an arbitrary interior value?
- 9 Why might a team deliberately choose not to pursue 100% branch coverage on every module?
- 10 How do CFG-based coverage criteria and the black-box techniques from the *Equivalence Partitioning* reading complement each other?

# Backup: References (from the Reading)

- ISTQB (2024). *Certified Tester Foundation Level (CTFL) Syllabus, v4.0.1*.
- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- Batchelder, N., et al. (2026). *coverage.py Documentation: Branch coverage measurement*. <https://coverage.readthedocs.io/en/latest/branch.html>
- pytest-cov contributors (2026). *pytest-cov Documentation*. <https://pytest-cov.readthedocs.io/>
- Apple Inc. (2014). *About the security content of iOS 7.0.6*. <https://support.apple.com/en-us/HT6147>
- Hasan, K. (2026). *Software Testing*. Grand Valley State University. <https://github.com/mdkamrulhasan/software-testing>